YAPAY ZEKÂYA GİRİŞ

9. ÜNİTE

BİRİNCİ DERECEDEN MANTIKTA ÇIKARIM

Inference in First-Order Logic

9.1–9.5 • Unification • Forward / Backward Chaining • Resolution

Kaynak: Russell & Norvig, Artificial Intelligence: A Modern Approach, 3. baskı

Bu ünitede neler öğreneceğiz?

Önerme mantığından genel birinci dereceden çıkarıma doğru ilerleyen beş başlık.

9.1	**Niceleyiciler**	UI / EI, önermeselleştirme
9.2	**Unification**	Yerine koyma, MGU, lifting
9.3	**İleri zincirleme**	Bilinenlerden yeni gerçek üretme
9.4	**Geri zincirleme**	Hedeften başlayarak kanıtlama
9.5	**Resolution**	CNF, çürütme, tamlık

Önerme mantığı ve birinci dereceden mantık

Çıkarımın amacı: KB ⊨ α ise α'yı kanıtlamak.

Önerme mantığı

P: "Ali öğrencidir."
Q: "Ali sınava girer."

$P \Rightarrow Q, P \implies Q$

Nesnelerin iç yapısı görünmez.

Birinci dereceden mantık (FOL)

$\forall x\ Student(x) \Rightarrow TakesExam(x)$
$Student(Ali)$

$\implies TakesExam(Ali)$

Nesne, ilişki, değişken, niceleyici vardır.

Sağlamlık: yanlış sonuç üretmeme • Tamlık: gerektirilen her sonucu ispatlayabilme

Universal Instantiation (UI)

Evrensel kuraldaki değişkenin yerine ground terim koyma.

$$\forall x\,[King(x) \wedge Greedy(x) \Rightarrow Evil(x)]$$

x yerine John konur → {x/John}

$$King(John) \wedge Greedy(John) \Rightarrow Evil(John)$$

Kritik nokta

UI yalnızca koşullu ifadeyi özelleştirir. Evil(John) için ayrıca King(John) ve Greedy(John) gerekir.

Existential Instantiation (EI)

Var olduğu bilinen, kimliği bilinmeyen nesneye yeni sabit adı verme.

$$\exists x\,[\mathrm{Crown}(x) \wedge \mathrm{OnHead}(x, \mathrm{John})]$$

Yeni Skolem sabiti: C_1

$$\mathrm{Crown}(C_1) \wedge \mathrm{OnHead}(C_1, \mathrm{John})$$

UI

∀x → herhangi bir ground terim; tekrar uygulanabilir.

EI

∃x → daha önce kullanılmamış bir sabit; aynı varlık için yeni tanık.

Önermeselleştirme (Propositionalization)

Evrensel kuralı tüm ground örneklere ayırarak önerme mantığı uygulama.

Bilgi tabanı

∀x: King(x) ∧ Greedy(x) ⇒ Evil(x)
King(John)
Greedy(John)
Brother(Richard,John)

Ground örnekler

King(John) ∧ Greedy(John) ⇒ Evil(John)

King(Richard) ∧ Greedy(Richard) ⇒ Evil(Richard)

John için sonuç: Evil(John)

Richard’ın kral ve açgözlü olduğunu bilmiyoruz: Evil(Richard) sonucunu çıkaramayız.

Sonsuz grounding ve yarı karar verilebilirlik

Fonksiyon sembolleri, sınırsız derinlikte ground terim üretebilir.

John (derinlik 0)

Father(John) (derinlik 1)

Father(Father(John)) (derinlik 2)

Father(Father(Father(John))) (derinlik 3)

Herbrand teoremi ve sınır

İspat varsa sonlu ground altküme yeterlidir; ispat yoksa arama sonsuza dek sürebilir.

Lifting: Genelleştirilmiş Modus Ponens

Tüm örnekleri üretmek yerine yalnızca gereken değişken eşleştirmelerini yap.

$$King(x) \wedge Greedy(x) \Rightarrow Evil(x)$$

$$King(John) \quad + \quad Greedy(y)$$

Birleştirici: θ = {x/John, y/John}

$$\textbf{Sonuç: } Evil(John)$$

Lifting: önermesel çıkarım kuralını değişkenli FOL düzeyine taşıma.

Unification: ifadeleri aynı hâle getirme

UNIFY(p,q)=θ; SUBST(θ,p)=SUBST(θ,q) olacak şekilde θ bulunur.

Sorgu	Bilinen ifade	Birleştirici
Knows(John,x)	Knows(John,Jane)	{x/Jane}
Knows(John,x)	Knows(y,Bill)	{x/Bill, y/John}
Knows(John,x)	Knows(y,Mother(y))	{y/John, x/Mother(John)}
Knows(John,x)	Knows(x,Elizabeth)	İsim çakışması → başarısız

Farklı kurallardaki bağımsız değişkenler önceden yeniden adlandırılmalıdır.

Standardizing Apart ve MGU

Önce değişken çakışmasını önle, sonra en az kısıtlayıcı birleştiriciyi seç.

Standardizing Apart

Knows(John,x)
Knows(x,Elizabeth)

İkinci x → x_{17}

θ={x/Elizabeth, x_{17}/John}

Most General Unifier (MGU)

Knows(John,x)
Knows(y,z)

MGU: {y/John, x/z}

{x/John, y/John, z/John} daha özel.

MGU, yalnızca zorunlu eşitlikleri koyar; sonraki çıkarımlar için esneklik sağlar.

Occur Check ve UNIFY algoritması

Bir değişken kendisini içeren terime bağlanamaz (standart sonlu-terim mantığı).

Neden Occur Check?

UNIFY(P(x), P(f(x)))

Gerekli koşul: x = f(x)

Yerine koyarsak:
x = f(f(x)) = f(f(f(x))) = ...

Sonuç: FAILURE

UNIFY işleminin ana adımları

1. İfadeler eşit mi?
2. Değişkense uygun bağlama yap
3. Fonksiyon simgelerini karşılaştır
4. Argümanları sırayla birleştir
5. Tutarsız bağlama veya occur check?
6. Başarı: θ / Başarısızlık: fail

STORE / FETCH ve yüklem indeksleme

Bilgi tabanında sorguyla eşleşme ihtimali olan tümcelere hızlı erişim.

STORE(s) / FETCH(q)

STORE: tümceyi bilgi tabanına ekler.

FETCH: sorguyla birleştirilebilen gerçeklere ait birleştiricileri bulur.

FETCH(Knows(John,x))

Predicate Indexing

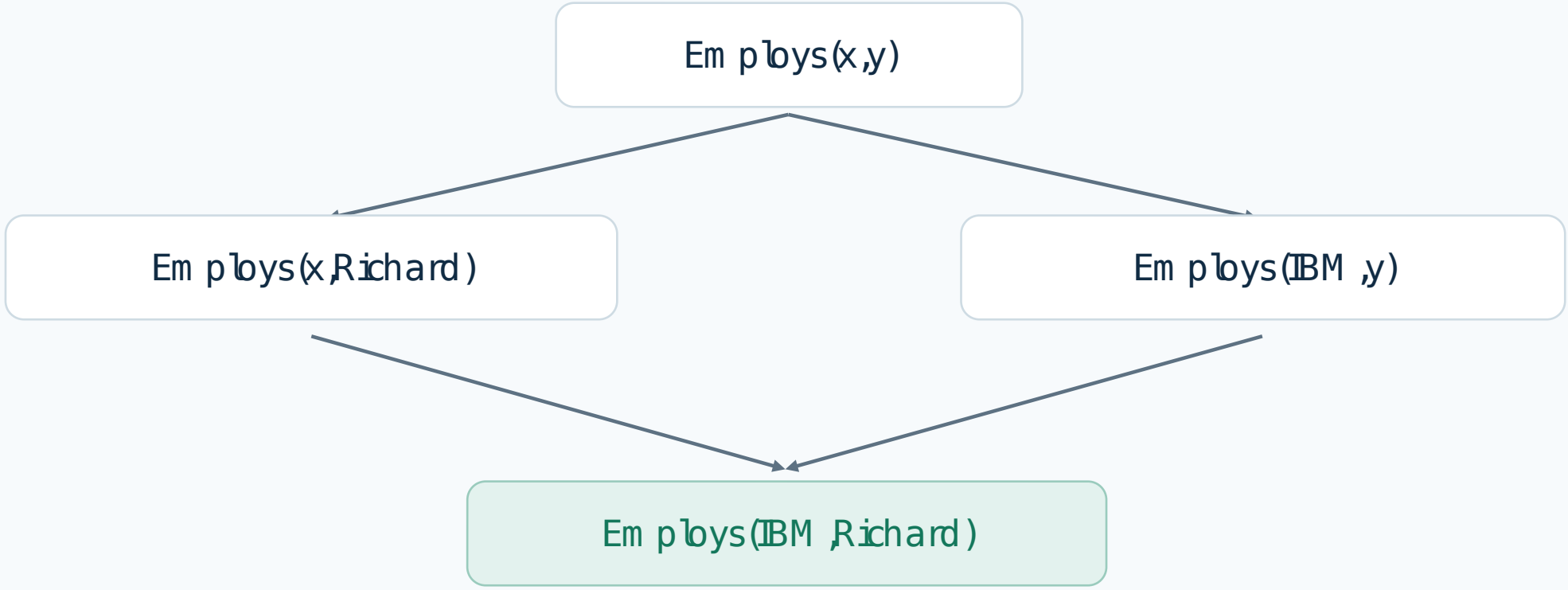
Knows(...) kayıtları bir grupta
Brother(...) kayıtları başka grupta
Employs(...) için argüman indeksleri

Örnek anahtar: (Employs, Richard)

Daha çok indeks = daha hızlı sorgu, ancak daha yüksek depolama ve güncelleme maliyeti.

Subsumption Lattice: genelden özele

Aynı kayıt farklı sorgu biçimleriyle erişilebilir; özgül sorgular değişkenleri sabitler.



Daha aşağı = daha fazla argüman belirlenmiş; IBM–Richard en özel örnektir.

Forward Chaining: definite clause ve Datalog

İleri zincirleme başlangıç gerçeklerinden hareket ederek uygulanabilir kuralları çalıştırır.

Definite Clause

King(x) ∧ Greedy(x) ⇒ Evil(x)

King(John)

Öncül: pozitif literallerin VE'si.
Sonuç: tek pozitif literal.

Datalog

Fonksiyon sembolü içermeyen definite clause bilgi tabanı.

Sonlu sabit kümesi olduğunda olası ground atom sayısı sonludur.

İleri zincirleme sabit noktaya varır.

Kural biçimi: Situation ⇒ Response • Yeni bilgi geldiğinde kurallar etkinleşebilir.

Suç örneği: bilgiyi mantığa dönüştürme

Amaç: Criminal(West) sonucunu çıkarabilmek.

Başlangıç gerçekleri

Owns(Nono,M_1)
Missile(M_1)
American(West)
Enemy(Nono,America)

M_1: Nono'nun sahip olduğu bir füze.

Çıkarım kuralları

1 American(x) ∧ Weapon(y) ∧ Sells(x,y,z) ∧ Hostile(z) ⇒ Criminal(x)

2 Missile(x) ∧ Owns(Nono,x) ⇒ Sells(West,x,Nono)

3 Missile(x) ⇒ Weapon(x)

4 Enemy(x,America) ⇒ Hostile(x)

FOL-FC-ASK: basit algoritma

Öncülleri karşılanan bütün kurallar, yeni atomik gerçekler üretir.

ADIM 1

Eşleştir

Bilgi tabanındaki gerçekleri kuralların öncülleriyle UNIFY et.

ADIM 2

Yeni bilgi ekle

SUBST(θ,q) sonucunu üret; zaten biliniyorsa tekrar ekleme.

ADIM 3

Tekrarla

Sorgu cevaplanınca veya new boş kalınca dur.

Kavramsal olarak basit, ancak her turda bütün kuralları kontrol ettiği için verimsizdir.

West örneğinde iki iterasyon

Her yeni gerçek, sonraki turda başka bir kuralı etkinleştirebilir.

1. iterasyon

Missile(M_1) + Owns(Nono,M_1)
→ Sells(West,M_1,Nono)

Missile(M_1)
→ Weapon(M_1)

Enemy(Nono,America)
→ Hostile(Nono)

2. iterasyon

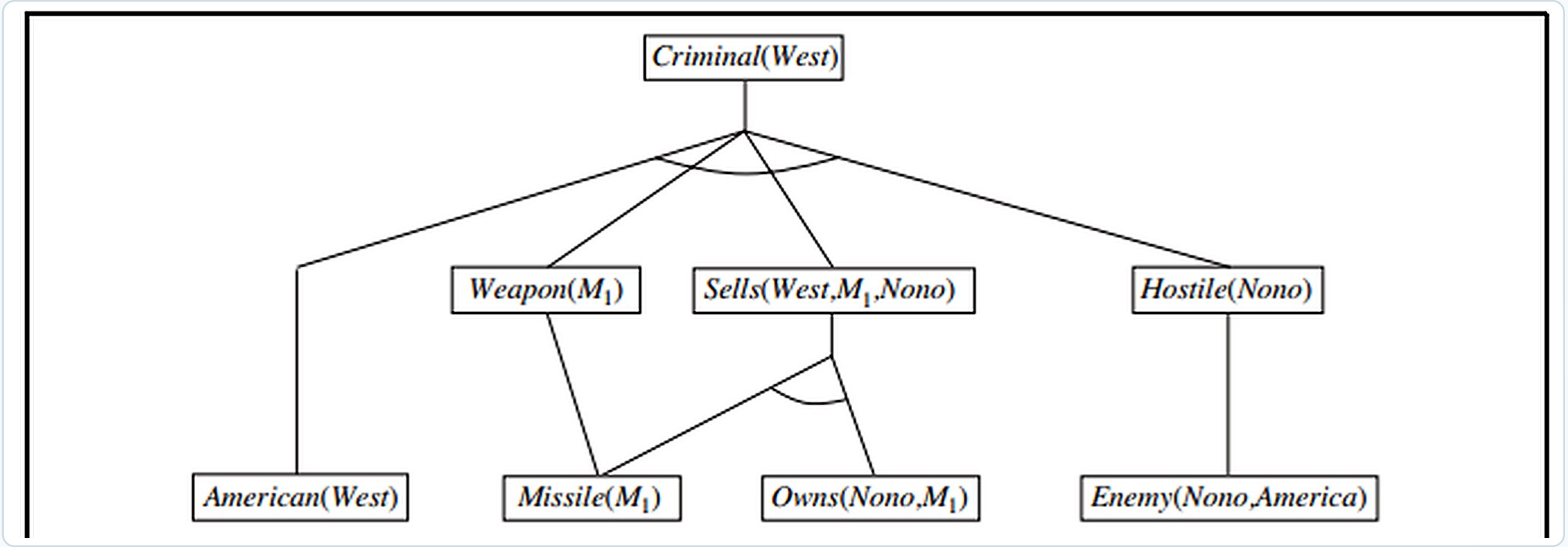
American(West) ✓
Weapon(M_1) ✓
Sells(West,M_1,Nono) ✓
Hostile(Nono) ✓

θ = {x/West, y/M_1, z/Nono}

Criminal(West)

Şekil 9.4 — İleri zincirleme ispat ağacı

Kitabın orijinal şeması: aşağıdan yukarıya doğru okunur.



Alt → orta → üst seviye

En alt: başlangıç gerçekleri. Orta: Weapon(M_1), Sells(West,M_1,Nono), Hostile(Nono). Üst: Criminal(West).

Şekildeki bağlantılar, her yeni çıkarımın hangi gerçeklere ve kurallara dayandığını gösterir.

Sabit nokta, sağlamlık ve tamlık

Bir bilgi tabanı daha fazla genişlemiyorsa çıkarımın sabit noktasına ulaşılmıştır.

Fixed Point

new = ∅ olduğunda durulur.

Definite clause çıkarımı sağlamdır.

Datalog'da fonksiyon sembolü yoktur: olası ground gerçek sayısı sınırlı.

Sonluluk ve sınır

p: yüklem sayısı
n: sabit sayısı
k: en büyük arite

Üst sınır: $p \cdot n^k$

Fonksiyon sembolü varsa sonsuz sayıda yeni gerçek üretilebilir.

Verimli eşleştirme: koşul sırası

Conjunct Ordering, toplam arama maliyetini azaltacak öncül sırasını arar.

$$Missile(x) \wedge Owns(Nono, x) \Rightarrow Sells(West, x, Nono)$$

Kötü sıra

Nono'ya ait 10.000 nesnenin her birini kontrol et; füze mi diye bak.

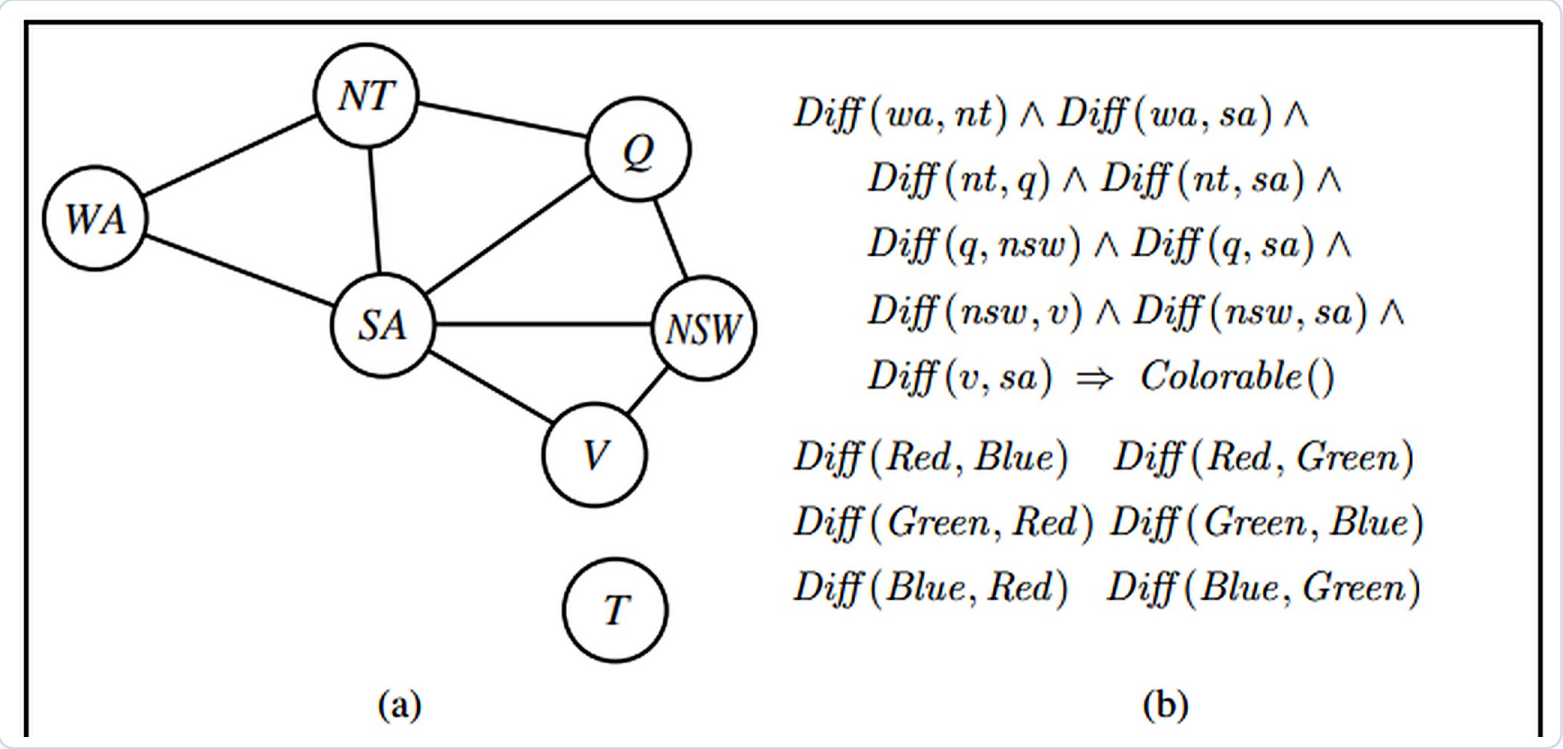
İyi sıra

Önce yalnızca 20 füzeyi bul; Nono'nun sahip olup olmadığını kontrol et.

MRV sezgiseli: daha az adayı olan koşulu önce kontrol etmek genellikle yararlıdır.

Şekil 9.5 — CSP ve kural eşleştirme

Harita boyama problemi, bir definite clause ve Diff gerçekleriyle temsil edilebilir.



Solda kısıt grafiği • Sağda mantıksal kurallar

WA, NT, SA, Q, NSW, V bölgelerine Red / Green / Blue atanır. Her kenar, komşuların farklı renkli olması gerektiğini söyler.

Colorable() ancak bütün Diff koşulları aynı anda karşılanırsa çıkarılabilir.

Incremental Forward Chaining ve Rete

Her iterasyonda bütün kuralları yeniden taramak yerine değişen bilgileri takip et.

Artımlı ileri zincirleme

t turunda yeni çıkarılan her gerçek, t−1 turunda eklenen en az bir yeni gerçeğe dayanır.

Kural ancak yeni bir gerçekle öncül eşleşiyorsa yeniden denenir.

Rete ağı

Kural öncüllerine ait kısmi eşleşmeleri bellekte saklar.

Yeni veri geldiğinde eksik koşulları tamamlar; aynı hesaplamayı baştan yapmaz.

Üretim sistemlerinde yaygındır.

Hedefle ilgisiz çıkarımları azaltma

Magic Sets, ileri zincirlemeyi belirli sorguya odaklamak için kuralı yeniden yazar.

$$American(x) \wedge Weapon(y) \wedge Sells(x,y,z) \wedge Hostile(z) \Rightarrow Criminal(x)$$

Hedef: Criminal(West) → Magic(West) ekle

$$Magic(x) \wedge American(x) \wedge Weapon(y) \wedge Sells(x,y,z) \wedge Hostile(z) \Rightarrow Criminal(x)$$

Kazanç

West dışındaki Amerikalılarla ilgili, hedefe katkı vermeyen çıkarımlar büyük ölçüde azaltılır.

Backward Chaining: hedeften geriye gitme

Alternatif kurallar OR; tek bir kuralın bütün öncülleri AND ilişkisi oluşturur.

Hedef odaklı arama

Sor: Pass(Ali)?

Pass(x) ← Student(x) ∧ Studies(x)

Alt hedefler:
Student(Ali)?
Studies(Ali)?

İkisi de doğruysa Pass(Ali).

AND / OR ayrımı

C ← A ∧ B
C ← D

OR: C'yi iki farklı kural kanıtlayabilir.

AND: birinci kural seçilirse A ve B birlikte ispatlanır.

FOL-BC-ASK algoritması

Derinlik öncelikli, soldan sağa, değişken eşleştirmelerini biriktiren arama.

FOL-BC-OR

1 Hedefi sonuç olarak içeren kuralları getir.
2 Değişkenleri yeniden adlandır.
3 Sonucu hedefle birleştir.
4 Sol taraftaki koşulları FOL-BC-AND ile kanıtla.

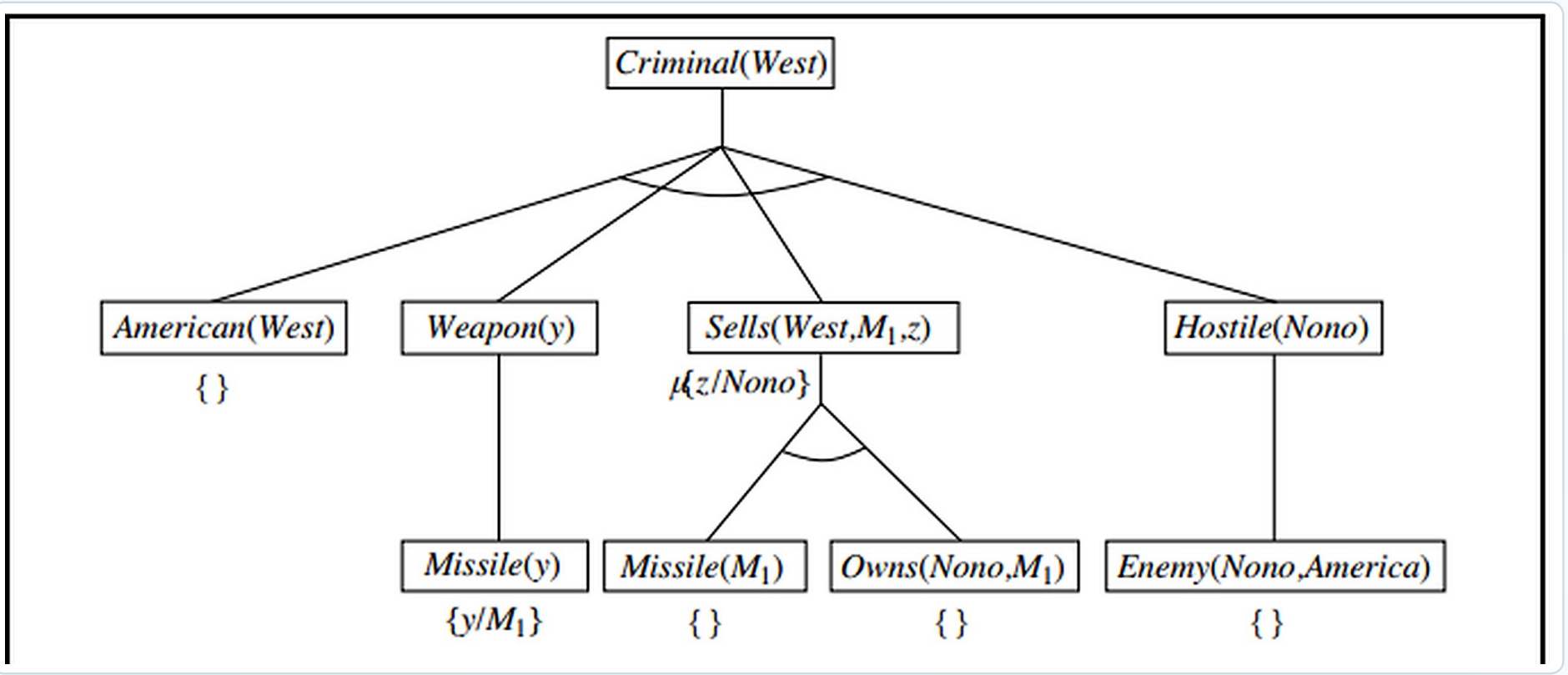
FOL-BC-AND

1 Bağlama başarısızsa geri dön.
2 Hedef kalmadıysa θ sonucunu üret.
3 İlk alt hedefi kanıtla.
4 Elde edilen θ'ları kalan alt hedeflere uygula.

Bir hedefin birden fazla geçerli cevabı olabilir: algoritma bir generator olarak tasarlanır.

Şekil 9.7 — Geri zincirleme ispat ağacı

Criminal(West) hedefine yukarıdan aşağıya, derinlik öncelikli yaklaşım.



Hedef: Criminal(West) → dört öncül

American(West) doğrudan gerçek. Weapon(y) → Missile(y) ile y=M_1. Satış kuralı z=Nono üretir; son hedef Hostile(Nono) olur.

Prolog: mantık + kontrol

Prolog, definite clause ifadelerini “sonuç :- koşullar” biçiminde yazar.

Algorithm = Logic + Control

Standart FOL

American(x) ∧ Weapon(y) ∧
Sells(x,y,z) ∧ Hostile(z)
⇒ Criminal(x)

Prolog

```
criminal(X) :-
  american(X), weapon(Y),
  sells(X,Y,Z), hostile(Z).
```

Büyük harf: değişken

Prolog’da APPEND bir ilişki tanımlar

Aynı iki kural hem birleştirme yapabilir hem de sonuca göre olası girdileri bulabilir.

```
append([],Y,Y).
append([A|X],Y,[A|Z]) :- append(X,Y,Z).
```

Normal yön

append([1],[2],Z).

Z = [1,2]

Ters sorgu

append(X,Y,[1,2]).

([], [1,2]) / ([1],[2]) / ([1,2],[])

Choice Point, Trail ve derlenmiş Prolog

Alternatif dalları denemek ve değişken bağlamalarını geri almak için özel veri yapıları.

Choice Point

Bir hedef birden çok kuralla eşleşirse henüz denenmemiş seçenekleri saklar.

Backtracking yapılacak konumu hatırlar.

Trail

Değişken bağlamalarını kaydeder.

Başarısız dalda bağlamaları geri alarak başka alternatifin denenmesini sağlar.

WAM

Warren Abstract Machine: Prolog için ara talimat mimarisi.

Derlenmiş programlarda yorumlayıcı maliyetini azaltır.

OR ve AND paralelliği

Paralelleştirme mantıksal aramanın farklı düzeylerinde mümkündür.

OR-Parallelism

C ← A
C ← B

A üzerinden C ve B üzerinden C ispatları birbirinden bağımsız dallarda yürütülebilir.

Alternatiflerin paralel aranması.

AND-Parallelism

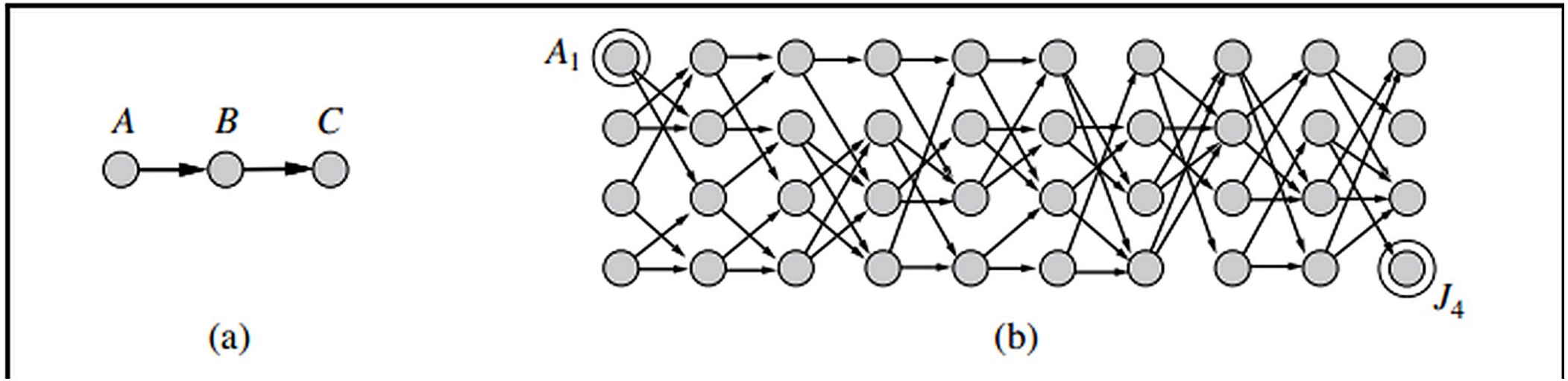
C ← A ∧ B

A ve B alt hedefleri paralel incelenebilir.

Ancak paylaşılan değişkenlerin eşleştirmeleri tutarlı olmalıdır.

Şekil 9.9 — Yol bulma ve arama maliyeti

Yol varlığı doğru olsa bile Prolog'un denetim sırası sonsuz aramaya neden olabilir.



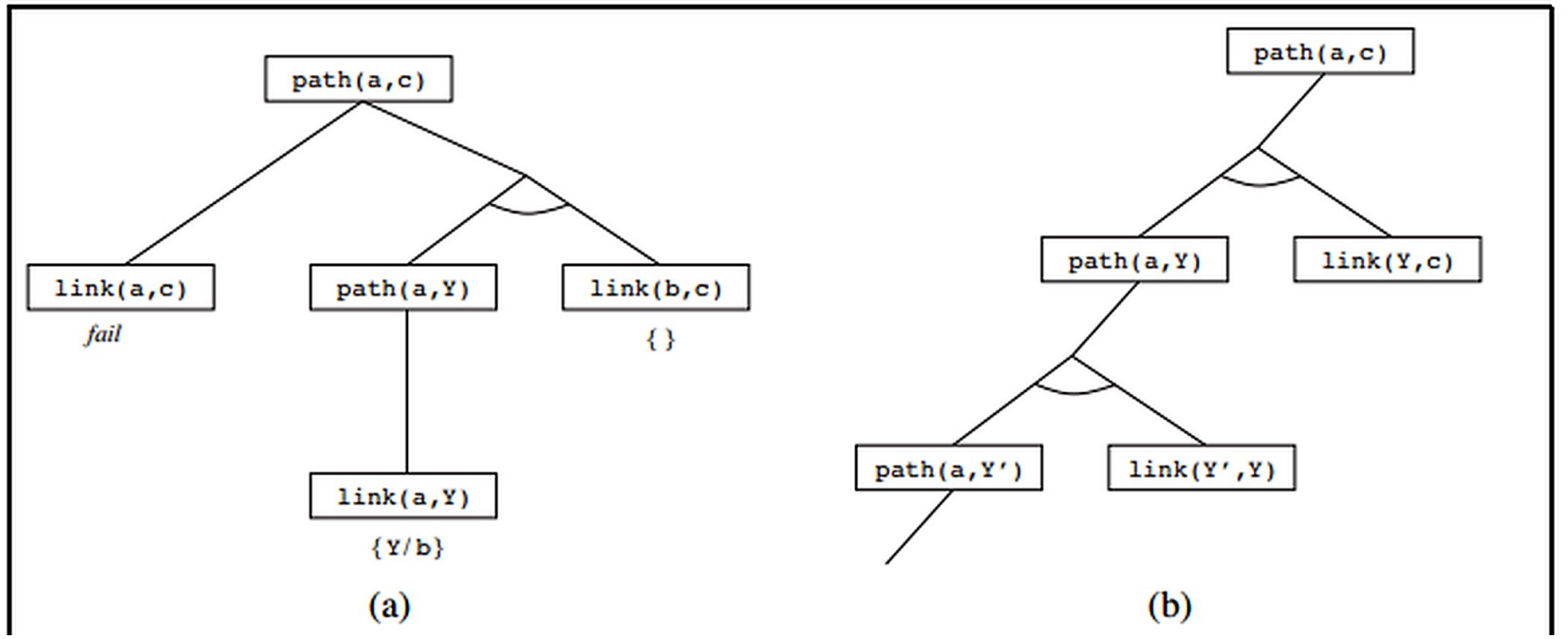
(a) A → B → C • (b) katmanlı yönlü grafik

(a) path(a,c) doğrudur. (b) A_1'den J_4'e hedefli derinlik öncelikli arama, tekrarlanan alt yollar yüzünden 877 çıkarım yapar.

Kural sırası ve arama stratejisi, mantıksal doğruluktan ayrı bir kontrol problemidir.

Şekil 9.10 — Başarılı ispat ve sonsuz özyineleme

Yalnızca iki Prolog kuralının sırası değişince davranış tamamen farklılaşabilir.



(a) Sonlu ispat • (b) Sonsuz ispat ağacı

Önce link(X,Z) denenirse path(a,b) bulunur, ardından link(b,c) kullanılır. Önce özyineleme denenirse path(a,Y) kendisini çağırabilir.

Önerilen kural sırası: önce doğrudan link, sonra özyinelemeli path kuralı.

Tekrarlanan çıkarımlar: Memoization ve Tabling

Daha önce çözülmüş alt hedefleri saklamak, aynı işi tekrar yapmayı önler.



Tabled Logic Programming

Geri zincirlemenin hedef odaklılığını, dinamik programlamanın önbellekleme verimliliğiyle birleştirir.

Prolog’un veritabanı semantiği

Unique Names ve Closed World varsayımları, standart FOL semantiğinden farklıdır.

Unique Names Assumption

Farklı sabit adları farklı nesneler olarak alınır.

CS ≠ EE
101 ≠ 102 ≠ 106

Course(CS,101) vb. dört kayıt ayrı dersleri gösterir.

Closed World Assumption

KB’den çıkarılamayan ifade başarısız kabul edilir.

Standart FOL ise belirtilmeyen olasılıkları açık bırakır.

Course(d,n) için completion, bilinen kayıtlarla ilişkiyi sınırlar.

Constraint Logic Programming (CLP)

Değişkene tek değer atamak yerine izin verilen değerleri kısıt olarak ifade etme.

$$\text{triangle}(3,4,Z) \quad X+Y \geq Z,\ Y+Z \geq X,\ X+Z \geq Y$$

ADIM 1

1. eşitsizlik

$3 + 4 \geq Z$
$\rightarrow Z \leq 7$

ADIM 2

2. eşitsizlik

$4 + Z \geq 3$
$\rightarrow Z \geq -1$

ADIM 3

3. eşitsizlik

$3 + Z \geq 4$
$\rightarrow Z \geq 1$

Sonuç: $1 \leq Z \leq 7$

Resolution: çelişki yoluyla ispat

Genel FOL tümceleri için, tamamlayıcı literalleri birleştirerek boş tümceye ulaşma.

$$KB \models \alpha \quad \Leftrightarrow \quad KB \wedge \neg\alpha \text{ tatmin edilemez}$$

ADIM 1

Olumsuzla

Kanıtlanacak α için ¬α ifadesini bilgi tabanına ekle.

ADIM 2

Çözümle

CNF tümcelerindeki tamamlayıcı literalleri UNIFY ederek ele.

ADIM 3

Çelişki

Boş tümce (□) bulunursa α kanıtlanır.

Conjunctive Normal Form (CNF)

Her tümce literallerin VEYA'sıdır; tümceler birbirine VE ile bağlanır.

$$(A \vee B) \wedge (\neg C \vee D)$$

Literal ve tümce

Literal: P(x) veya ¬P(x)

Tümce: A ∨ ¬B ∨ C

Dönüşüm örneği

$A \wedge B \Rightarrow C$
$\equiv \neg(A \wedge B) \vee C$
$\equiv \neg A \vee \neg B \vee C$

CNF dönüşümünün altı adımı

Standart işlem sırası, Skolemleştirme dışında önerme mantığındaki dönüşüme benzer.

1 ⇒ ve ⇔ bağlaçlarını kaldır

2 ¬ işaretini niceleyicilerin içine taşı

3 Değişken isimlerini ayır

4 ∃ niceleyicilerini Skolemleştir

5 ∀ niceleyicilerini görünümden çıkar

6 ∨ işlemini ∧ üzerine dağıt

Skolem sabiti mi, fonksiyonu mu?

Varoluşsal değişken, kapsamındaki evrensel değişkenlere bağlı olabilir.

Skolem Sabiti

∃y Loves(John,y)

→ Loves(John,A)

A yeni bir sabittir.

Evrensel değişkene bağımlılık yoktur.

Skolem Fonksiyonu

∀x ∃y Loves(x,y)

→ Loves(x,F(x))

F(x), x'e bağlı tanığı temsil eder.

Her x için tanık farklı olabilir.

Skolemization aynı modelleri değil, tatmin edilebilirliği korur.

CNF örneği: bütün hayvanları seven kişi

“Bütün hayvanları seven herkes, biri tarafından sevilir.”

$$\forall x\,[(\forall y\,Animal(y) \Rightarrow Loves(x,y)) \Rightarrow \exists z\,Loves(z,x)]$$

- Koşulluları kaldır; olumsuzlukları içeri taşı.
- Değişkenleri ayır, ∃y ve ∃z için F(x) ve G(x) üret.
- ∀ niceleyicilerini kaldır; ∨’yi ∧ üzerine dağıt.

$$Animal(F(x)) \vee Loves(G(x),x)$$

$$\neg Loves(x,F(x)) \vee Loves(G(x),x)$$

FOL Resolution, Unification ve Factoring

Bir literal, diğerinin olumsuzuyla birleştirilebiliyorsa çözümleme yapılır.

$$Animal(F(x)) \vee Loves(G(x), x)$$

$$\neg Loves(u, v) \vee \neg Kills(u, v)$$

θ = {u/G(x), v/x}

$$\mathbf{Animal(F(x)) \vee \neg Kills(G(x), x)}$$

Factoring: P(x) ∨ P(Ali), θ={x/Ali} ⇒ P(Ali).

Resolution ile basit çürütme ispatı

Bilgi tabanı Student(Ali) ve Student(x) ⇒ Studies(x) içersin.

$$\neg \text{Student}(x) \vee \text{Studies}(x)$$

$$\neg \text{Studies}(\text{Ali}) \quad \text{(olum suzlanm ış hedef)}$$

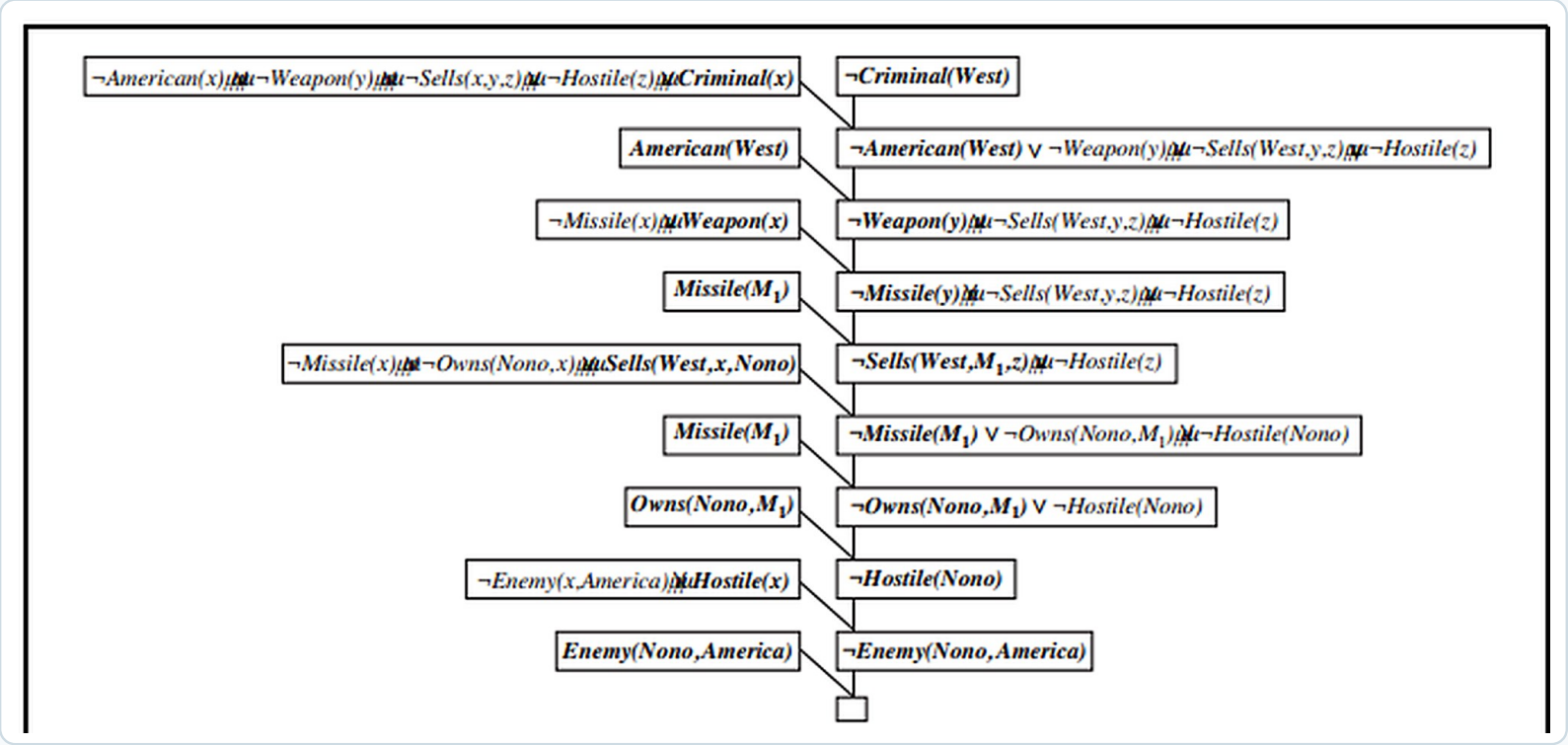
Resolution ve θ={x/Ali} → ¬Student(Ali)

$$\text{Student}(\text{Ali}) + \neg \text{Student}(\text{Ali}) \Rightarrow \square$$

Boş tümce bulundu: KB ⊨ Studies(Ali).

Şekil 9.11 — West için Resolution ispatı

Hedefin olumsuzlamasından başlanır ve bilgi tabanıyla çözümleme yapılır.



Kitap şekli: tamamlayıcı literaller kalın belirtilmiştir.

Negated goal: ¬Criminal(West). Ardışık çözümleme ile American, Weapon, Sells ve Hostile olumsuzlukları elenir; en sonda □ bulunur.

Bu omurga yapısı, Horn tümceleri üzerinde geri zincirleme ile yakından ilişkilidir.

West için Resolution: ispat omurgası

Her yeni tümce bir öncekiyle KB'deki başka bir tümcenin çözümlemesidir.

¬Criminal(West)	Suç kuralını kullan
¬American(West) ∨ ¬Weapon(y) ∨ ¬Sells(…) ∨ ¬Hostile(z)	American(West) ile ele
¬Weapon(y) ∨ ¬Sells(…) ∨ ¬Hostile(z)	Missile(x)⇒Weapon(x)
¬Missile(y) ∨ ¬Sells(…) ∨ ¬Hostile(z)	Missile(M_1) ile ele
¬Sells(West,M_1,z) ∨ ¬Hostile(z)	Satış ve Owns kuralları
¬Hostile(Nono)	Enemy(Nono,America) ⇒ Hostile(Nono)
¬Enemy(Nono,America) → □	Çelişki, Criminal(West) ispatlandı

Curiosity ve Tuna: genel FOL ispatı

Bu örnekte definite clause olmayan kurallar ve Skolem fonksiyonları da bulunur.

Verilen bilgiler

A Bütün hayvanları seven sevilir.
B Hayvan öldüreni kimse sevmez.
C Jack bütün hayvanları sever.
D Jack veya Curiosity, Tuna'yı öldürdü.
E Cat(Tuna)
F Cat(x) ⇒ Animal(x)

Kanıtlanacak hedef

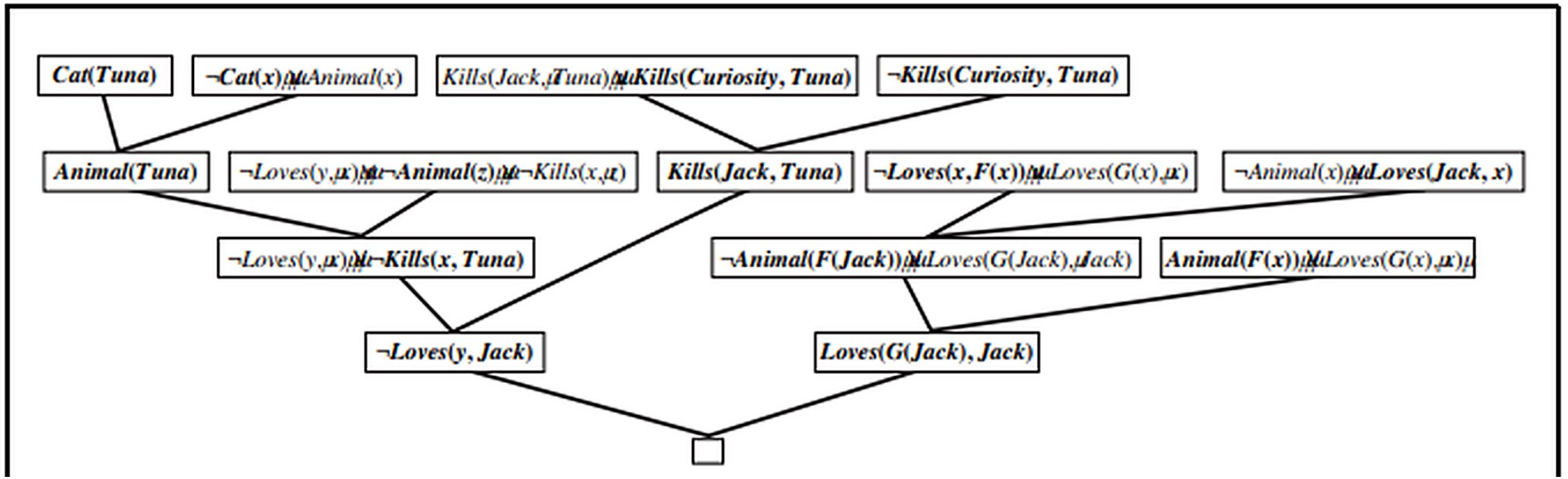
Kills(Curiosity,Tuna)?

Çürütme için:
¬Kills(Curiosity,Tuna)

Şimdi iki zıt sonuç aranır:
— Jack'i hiç kimse sevmez.
— Jack'i birisi sever.

Şekil 9.12 — Curiosity’nin Tuna’yı öldürmesi

İspat ağacındaki iki kol karşıt Loves literallerine ulaşır.



Üstte başlangıç tümceleri, altta boş tümce □

Sol kol: ¬Kills(Curiosity,Tuna) ⇒ Kills(Jack,Tuna) ⇒ ¬Loves(y,Jack). Sağ kol: Skolem / factoring ⇒ Loves(G(Jack),Jack).

Son birleşim: ¬Loves(G(Jack),Jack) ve Loves(G(Jack),Jack) ⇒ □.

Curiosity ispatının yorumlanması

Aynı anda “Jack’i hiç kimse sevmez” ve “Jack’i birisi sever” sonucu çıkıyor.

1. ispat kolu

Curiosity öldürmedi varsay.

Jack öldürdü.
Tuna kedidir → hayvandır.

Hayvan öldüren sevilmez:
¬Loves(y,Jack)

2. ispat kolu

Jack bütün hayvanları sever.

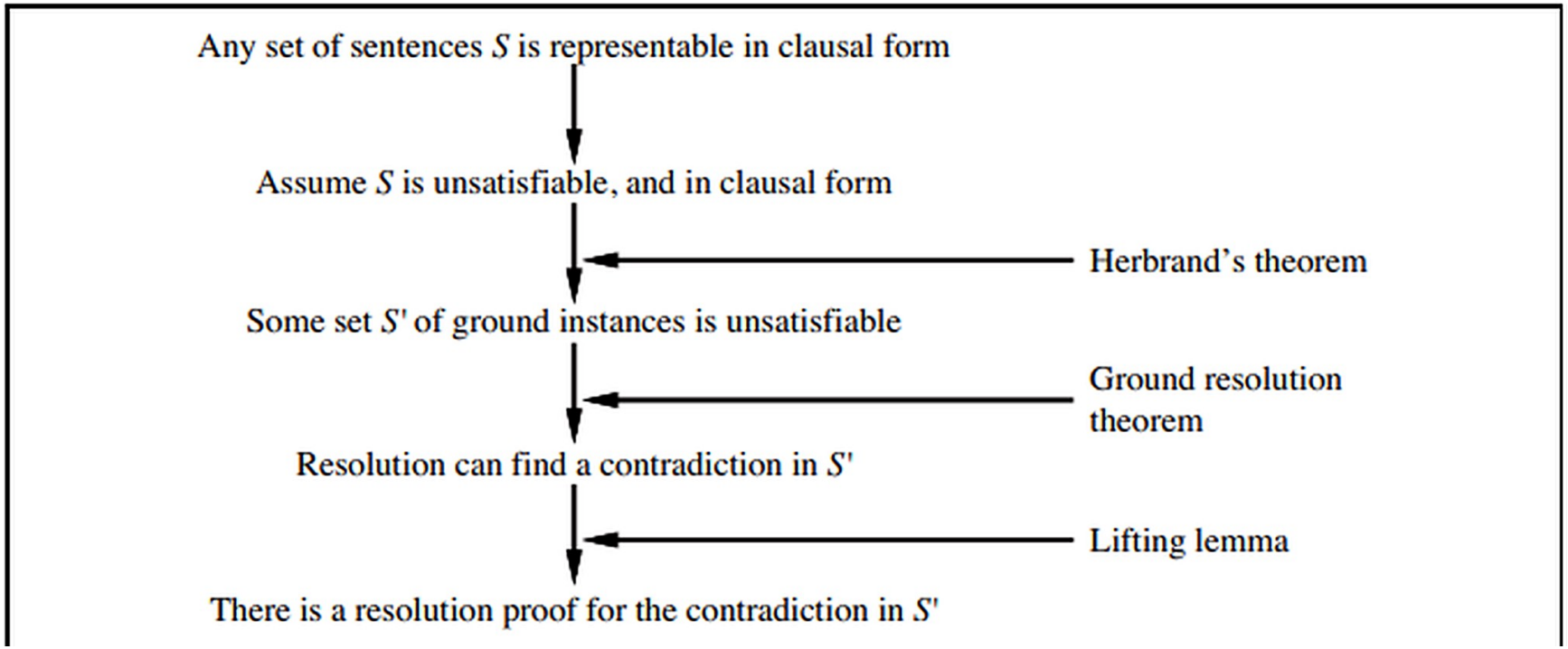
Herkesi seven en az biri tarafından sevilir.

Skolem ve factoring:
Loves(G(Jack),Jack)

Genel sorgular için Answer(w) literali kullanılarak somut değişken bağlaması döndürülebilir.

Şekil 9.13 — Resolution'ın tamlık ispatı

Herbrand → Ground Resolution → Lifting Lemma.



Tatmin edilemeyen S için sonlu bir çelişki ispatı vardır.

Herbrand sonlu tatmin-edilemeyen ground altküme verir; önerme resolution burada □ üretir; lifting aynı ispatı FOL düzeyine yükseltir.

Refutation-complete: S tatmin edilemezse sonunda sonlu sayıda çözümleme adımıyla □ elde edilir.

Herbrand Evreni ve Lifting Lemma

Sonsuz terim evrenine rağmen ispatta sonlu bir ground altküme yeterlidir.

Herbrand Universe

Sabitler: A, B
Fonksiyon: F(·,·)

H(S) = {A, B, F(A,A), F(A,B), F(B,A), ...}

Saturation: bütün uygun ground örnekleri üretme.

Lifting Lemma

Ground ispat adımı varsa, asıl değişkenli tümcelerde karşılığı da bulunabilir.

P(x) ∨ Q(x)
¬P(y) ∨ R(y)

⇒ Q(x) ∨ R(x)

Eşitlik: aksiyomlar ve Demodulation

Standart çözümleme, eşitlik hakkında ek aksiyomlar veya özel kurallar gerektirir.

Eşitlik aksiyomları

Reflexive: x=x

Symmetric: x=y ⇒ y=x

Transitive:
x=y ∧ y=z ⇒ x=z

Eşit terimler yüklem ve fonksiyonlarda yer değiştirebilir.

Demodulation

Father(Father(x))
 = PaternalGrandfather(x)

Birthdate(Father(Father(Bella)),1926)

⇒ Birthdate(PaternalGrandfather(Bella),1926)

Paramodulation ve çözümleme stratejileri

Eşitliği çözümlemede kullan; gereksiz çıkarımları uygun arama politikalarıyla sınırla.

Paramodulation örneği

$P(F(x,B),x) \vee Q(x)$
$F(A,y)=y \vee R(y)$

$\theta=\{x/A, y/B\}$

$\Rightarrow P(B,A) \vee Q(A) \vee R(B)$

Stratejiler

Unit Preference — kısa tümceye öncelik
Set of Support — hedefe ilgili tümceler
Input / Linear Resolution — ispat omurgası
Subsumption — gereksiz özel tümceleri ele

Uygulamalar: donanım/yazılım doğrulama, sentez.

Üç çıkarım yöntemi: aynı mantık, farklı arama

Ünitenin tamamında sağlamlık, tamlık ve verimlilik arasında ayrım yapıyoruz.

Yöntem	Nereden başlar?	Temel araç	Sınırlama / avantaj
Forward Chaining	Gerçekler	Genelleştirilmiş MP	Gereksiz sonuç üretebilir
Backward Chaining	Hedef	AND/OR + UNIFY	Hedef odaklı; döngü riski
Resolution	KB + ¬hedef	CNF + çözümleme	Genel FOL; refutation-complete

Kendini kontrol et

UI ve EI nasıl ayrılır? • MGU nedir? • Forward vs Backward? • Boş tümce neden ispat sayılır?